End-to-End Integration of a VLM with a Neuro-Symbolic State Manager for Embodied Robot Manipulation on TIAGo

Abderrahmene Amar Benhamlaoui[1 1], **Ferhat Attal**[1], **Imad Kenai**[1], **Abdelghani Chibani**[1], **Yacine Amirat**[1]

[1]LISSI Laboratory (EA 3956), Université Paris-Est Créteil (UPEC), Vitry-sur-Seine, France

abderrahmene.benhamlaoui@etu.u-pec.fr, ferhat.attal@u-pec.fr, imadkenai09@gmail.com, chibani@u-pec.fr, amirat@u-pec.fr

Abstract

Bridging natural language instructions and physical robot actions requires an agent that can reason about its environment, maintain a consistent world model, and execute a diverse range of skills — all without retraining for new objects or scenes. Existing approaches rely on fixed-vocabulary perception, static knowledge graphs that cannot reflect evolving scene state, or architectures that decouple high-level reasoning from low-level execution. We present a neuro-symbolic embodied agent for the TIAGo mobile manipulator in which a Vision-Language Model (Gemini 2.0 Flash) acts as the cognitive reasoning core — simultaneously perceiving open-vocabulary objects and selecting skills in a single inference step. A dynamic StateManager maintains a live semantic ontology of the scene: spatial relations between objects are inferred at every step and serialised as structured prompt context, grounding the agent's decisions in its current perceptual beliefs. A skill library of seven actions closes the loop from language to physical manipulation, including active visual search, RANSAC-based grasping, face-aware person identification, and object handover. Implemented and demonstrated on a real TIAGo, the agent operates without a local GPU and requires no retraining for new object categories.

Keywords: Embodied Agent, Vision-Language-Action, Neuro-Symbolic Robotics, Semantic Ontology, Open-Vocabulary Perception, TIAGo

1. Introduction

Building general-purpose assistive robots requires more than task-specific controllers: it requires an *embodied agent* that perceives open-ended scenes, maintains a belief about the world, reasons over that belief to choose appropriate actions, and physically executes them — all in a closed loop [1]. Recent Vision-Language Models (VLMs) and Vision-Language-Action (VLA) architectures offer a promising path toward this goal [2], yet real deployments expose three persistent gaps: *(a)* perception remains fixed-vocabulary, requiring retraining for new object categories; *(b)* the agent's world model is static — RDF graphs [4] and pre-encoded scene descriptions [5] cannot reflect the evolving scene state after each manipulation; *(c)* no existing GPU-free system closes the full agent loop from voice command to physical object handover on a real mobile manipulator.

This work is presented as a system integration and deployment demonstration rather than a novel algorithmic contribution. We show how a cloud VLM can be combined with a live symbolic state manager to form a practical embodied agent, and we validate the integration on a real TIAGo in a laboratory setting. The architecture directly extends the semantic framework of Eladdachi et al. [4], separating three layers: a *neural perception layer* (VLM), a *symbolic ontology layer* (StateManager), and an *action layer* (skill library). This paper demonstrates:

1. **A closed perception-reasoning-action loop.** The EmbodiedAgent integrates STT, a cloud VLM, a live state manager, and a skill library into a continuous agent cycle that runs without human intervention between commands.
2. **Live semantic state injection (StateManager).** Spatial relations (left-of, above, near, on-surface) are inferred from live detections, object memory persists across steps, and the full scene graph is serialised as structured context prepended to every VLM prompt — enabling multi-instance disambiguation without modifying VLM weights.
3. **Unified VLM for perception and planning.** A single Gemini 2.0 Flash call simultaneously detects open-vocabulary objects and selects the appropriate skill with rationale, avoiding the need for a separate detector and a separate planner.
4. **Complete action repertoire with depth-based localisation.** Seven skills (active head search, grasp, handover, wave, go-home, open/close-hand) are implemented and tested on real hardware; a foreground-cluster depth refinement improves 3D localisation from loose VLM bounding boxes without segmentation masks.

2. Related Work

VLM/VLA agentic architectures. SayCan [1] pioneered grounding LLM decisions with affordance value functions, demonstrating that language can drive behaviour at scale. ViLa [2] advances this by integrating visual observations directly into the LLM reasoning loop,

enabling closed-loop plan correction without external affordance modules — a key step toward agentic VLA systems. EMPOWER [3] instantiates a multi-role VLM planner on TIAGo with open-vocabulary grounding via YOLO-World and EfficientViT-SAM, achieving strong task success across six physical scenarios. Eladdachi et al. [4] ground a Gemini 1.5 Flash agent in a static RDF/OWL ontology on Pepper, but perception remains fixed-class and the knowledge graph cannot reflect post-action scene changes. Our integration addresses this limitation by pairing the VLM with a live symbolic state manager (StateManager) that maintains the agent's current belief and injects it as structured context at every inference step — effectively turning a stateless cloud API into a session-persistent embodied agent.

Neuro-symbolic agent architectures. Neuro-symbolic approaches explicitly separate neural perception from symbolic reasoning, giving rise to explainable, modular agents. Abdelkawy et al. [6] demonstrated that coupling spatio-temporal CNNs with NKRL ontology inference substantially improves activity-aware robot behaviour. Kenai et al. [7] combine VLMs with incremental learning and answer set programming for symbolic reasoning over unknown objects. Our architecture follows this two-layer pattern: the VLM provides neural perception grounded by a symbolic spatial-relation ontology (StateManager) that is updated at every inference step — a live belief state rather than a static knowledge base.

Semantic world models for HRI. SayPlan [5] uses large-scale scene graphs as spatial knowledge for long-horizon planning, but these graphs are pre-built and cannot adapt after manipulation. A key distinction of our approach is that the agent's world model is constructed *online* from live visual evidence, remaining consistent with the current scene state after each action.

3. System Architecture

The EmbodiedAgent (Fig. 1) implements a neuro-symbolic VLA architecture structured around three layers: *(i)* a **neural perception-reasoning layer** (VLMReasoner, Gemini 2.0 Flash) that processes RGB images and language to select actions; *(ii)* a **symbolic ontology layer** (StateManager) that maintains the agent's semantic world model and injects it into every VLM inference; and *(iii)* a **physical action layer** (skill library, ROS/MoveIt) that grounds language decisions in physical robot motions. On each agent cycle, a voice command is transcribed, the VLMReasoner queries the scene with the current ontological context from StateManager, object memory and spatial relations are updated, and the selected skill is dispatched. A safety subsystem monitors consecutive failures and triggers a safe-mode sequence after three errors, preventing hardware damage.

3.1 Speech Interface

Voice commands are transcribed using a dual-engine pipeline: a local Whisper model serves as the primary recogniser, with a cloud speech API as fallback when the local model is uncertain. The robot responds through the PAL TTS system, giving spoken feedback after each action.

3.2 VLM Reasoner (VLMReasoner)

VLMReasoner wraps Gemini 2.0 Flash and serves as the agent's cognitive core. Given a voice command u, the current camera image $\mathbf{I}$, and the semantic context ctx from StateManager, a single API call returns:

$$\mathcal{V}(u, \mathbf{I}, \text{ctx}) \longrightarrow \{\, \texttt{skill},\ \texttt{params},\ \texttt{rationale},\ D \,\} \quad (1)$$

where D is the set of detected objects with bounding boxes and labels. The model thus acts simultaneously as an open-vocabulary detector and a task planner — no separate perception module is required. The rationale ρ returned alongside the skill choice is preserved and used to seed the more specific object-grounding prompt at grasp time (Section 3.6), propagating the agent's intent downward through the execution stack.

3.3 Semantic World Model (StateManager)

StateManager is the agent's *semantic world model* — the symbolic layer of the neuro-symbolic architecture. It accumulates evidence from every VLM inference step and from the robot's own sensors (joint state and odometry), maintaining a live picture of what has been seen, where, and in what relation to other objects. This replaces the static, offline-built RDF graph of [4] with a belief state that evolves as the scene changes.

3.3.1 Object memory.

For every detected object class c, the StateManager maintains a structured memory entry $\mathcal{M}[c]$ recording: the last detected bounding box, the robot's odometric pose at the time of last sighting, the number of times the object has appeared in the session, the current confidence score, the set of active spatial relations, and a named location label. Entries older than five minutes are excluded from the VLM context, preventing the agent from acting on stale observations.

3.3.2 Automatic spatial relation inference.

For each pair of detected objects, the system computes the Euclidean distance between their bounding box centroids. When two objects are within 150 px of each other, spatial relations are automatically assigned: an object whose bottom edge sits above another's centre is labelled `above`; horizontal centroid ordering determines `left_of` and `right_of`; objects whose bottom edges are nearly level (within 50 px) are jointly tagged `on_table`. The full scene graph $\mathcal{G}$ is serialized as structured text and prepended to each VLM prompt:

Gripper: holding:bottle | Location: table_area | Relations:

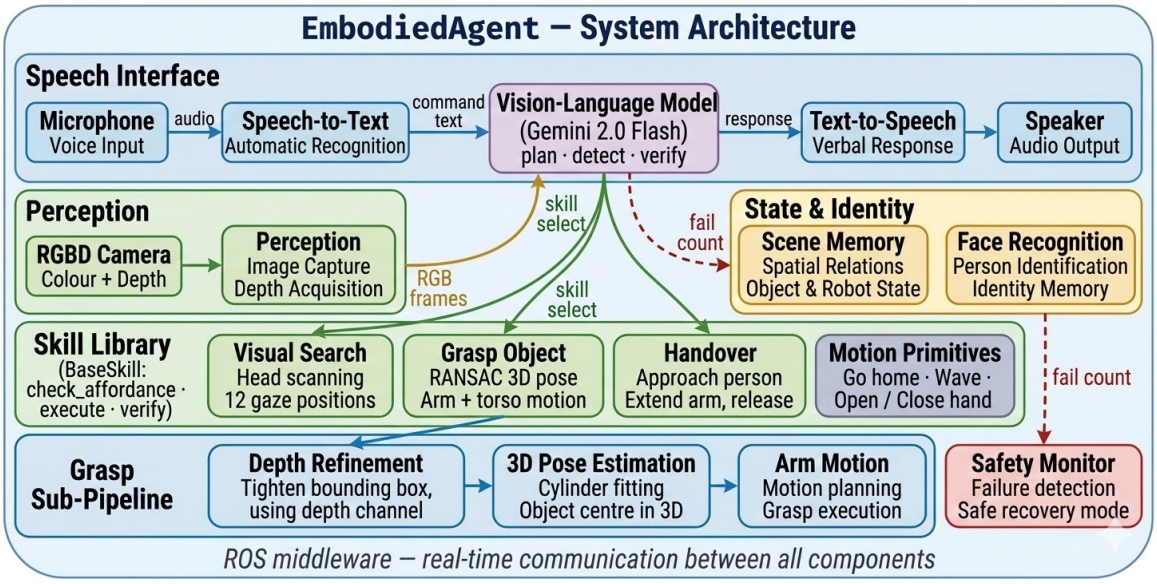

Figure 1: **System architecture.** The EmbodiedAgent orchestrates a continuous perception-reasoning-action loop: STT parses voice commands, the VLMReasoner (Gemini 2.0 Flash) simultaneously detects objects and selects skills, the StateManager injects live spatial context and updates its scene ontology, and the skill library executes on the TIAGo hardware via ROS. The FaceManager microservice runs on the host machine and communicates over HTTP (port 5002).

bottle left_of cup, cup near box | Memory: cup (3×, 12 s ago) | History: search_with_head✓

3.4 Face Recognition (FaceManager)

FaceManager runs as a separate microservice, decoupling face inference from the main ROS process. It encodes detected faces as compact embedding vectors and matches them against a session memory using Euclidean distance: a face is recognised if its distance to a stored embedding falls below a fixed threshold. When a known person is seen, the robot greets them by name. When an unfamiliar face appears, the agent initiates a brief introduction dialogue — it asks for the person's name, listens via STT, uses the VLM to parse the response, and stores the face for the rest of the session.

3.5 Active Visual Search (`search_with_head`)

When the target is not visible in the initial view, the active search skill (Fig. 2) systematically pans and tilts TIAGo's head through twelve positions covering the full workspace. After settling at each position, the VLM is queried for the target object; if found, the search ends and the detection is handed off to the grasping pipeline. For person targets, an additional check confirms that a face is visible in the upper portion of the detected bounding box — preventing false positives on feet or torsos — and adjusts the head tilt if needed.

3.6 Object Grasping Pipeline

The grasping pipeline (Fig. 3) takes the agent from language-level object reference to physical contact in four stages.

3.6.1 Context-grounded detection.

Before querying the VLM for a precise bounding box, the skill assembles a disambiguation prompt that combines the agent's original rationale ρ with the spatial relations currently held in StateManager:

> *"{class}. The operator selected: [rationale] | Spatial: [relations]. Return ONLY that one object."*

This grounds the detection in the agent's broader reasoning context, helping distinguish between multiple instances of the same class. The resulting bounding box is cached and reused across all depth frames to ensure a consistent object selection throughout pose estimation.

3.6.2 Depth-based bounding box refinement.

VLM bounding boxes are semantically accurate but spatially coarse — they often include table surface and background pixels. To address this, we isolate the nearest foreground depth cluster within the VLM box: depth pixels within a tolerance of 12 cm above the nearest point in the region are retained, and the refined bounding box is their axis-aligned enclosure. This produces a tight crop around the object without any segmentation model.

3.6.3 3D pose estimation.

Pixels within the refined box are back-projected to 3D using camera intrinsics and a coordinate transform to the robot base frame. RANSAC fits a vertical cylinder to the resulting point cloud, which works well for bottles and cans; flat or box-shaped objects fall back to a median centroid estimate. Five successive estimates are averaged

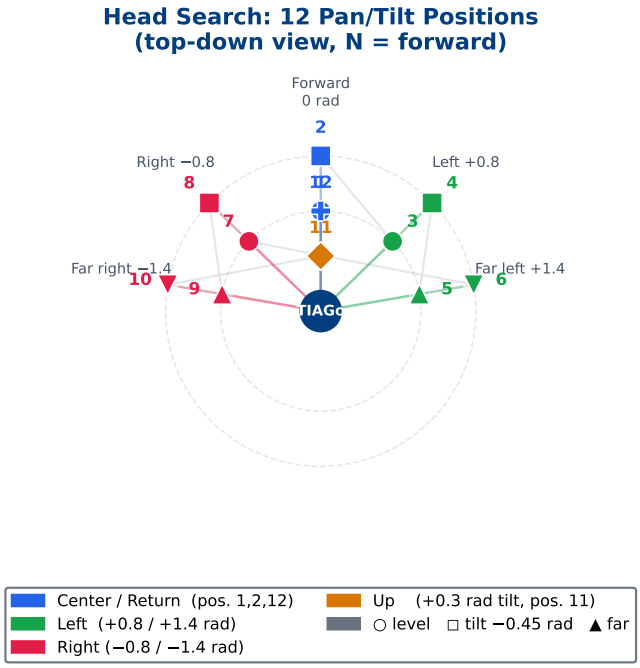

Figure 2: **Head scan pattern.** The robot visits twelve pan/tilt head positions sequentially, running a VLM detection query at each stop; search ends as soon as the target is found.

for stability, and a reachability check discards poses that fall outside the robot's arm workspace.

3.6.4 Arm execution.

MoveIt plans a collision-free trajectory to a pre-grasp pose 20 cm above the object. Final descent is performed by lowering the torso lift joint in a straight vertical line, which guarantees a top-down approach without IK divergence near the table surface. After grasping, the trunk rises to clear the workspace before the robot returns to its home pose.

3.7 Object Handover

The handover skill detects the person in the current camera frame, back-projects their bounding box centroid to 3D, and commands MoveIt to extend the arm toward the person at a fixed handover height of 0.85 m, capped at 0.62 m in depth to stay within the reachable workspace. Stale collision objects from the prior grasp are cleared before planning. The robot announces the handover via TTS, waits 4 s for the person to take the object, then opens the gripper and returns home.

4. Discussion

Neuro-symbolic agent loop. The key architectural insight is the separation of *neural* perception (VLM) from *symbolic* reasoning (StateManager): the VLM produces rich, open-vocabulary observations, while the StateManager maintains a structured, interpretable world model that constrains and grounds the next VLM call. This closed loop turns a stateless VLM API into a stateful, session-aware agent — capable of tracking what the robot is holding, what objects it has seen, and how they relate, across an entire interaction.

Live semantic ontology vs. static knowledge graphs. Unlike the static RDF graph of [4] — which requires offline construction and cannot reflect scene changes — the StateManager updates its scene ontology at every inference step from live visual evidence. This is critical in interactive manipulation: once an object is grasped, the StateManager immediately registers the change in gripper state and propagates it into the next prompt, preventing the agent from attempting to re-grasp an already-held object. The 5-minute recency window is configurable; a shorter timeout is preferable in highly dynamic environments.

VLM as unified cognitive module. Querying a single model for both visual grounding and skill selection eliminates the latency and error cascade of a detect-then-plan pipeline. The rationale returned alongside each skill choice acts as structured chain-of-thought that seeds the subsequent grasp-level disambiguation prompt, propagating the agent's high-level intent down to the physical execution layer — a lightweight form of hierarchical planning.

Grounding language in physical space. VLM bounding boxes are semantically accurate but spatially loose. The depth-based refinement step bridges the gap between language-level perception and physical-level manipulation: by isolating the nearest foreground cluster, it produces a tight bounding volume that grounds the agent's object reference in metric 3D space, without any segmentation model.

Computational footprint and API dependency. The entire inference pipeline runs without a local GPU, with all ROS control, RANSAC, and MoveIt planning executed on a standard laptop CPU. This lowers the hardware barrier for deployment on commodity service robots. The trade-off is a dependency on cloud API availability: Gemini 2.0 Flash round-trip latency is the primary bottleneck, and network outages represent a single point of failure. Deploying a quantised local VLM would eliminate this dependency at the cost of higher latency — a reasonable path for safety-critical or offline deployments.

5. Limitations and Future Work

The current system has several identifiable limitations. **Geometry model.** Cylindrical RANSAC fails on flat, irregular, or handle-bearing objects; future work could add a superquadric fitting fallback or learn object-class-specific grasp templates from a small offline dataset. **Static**

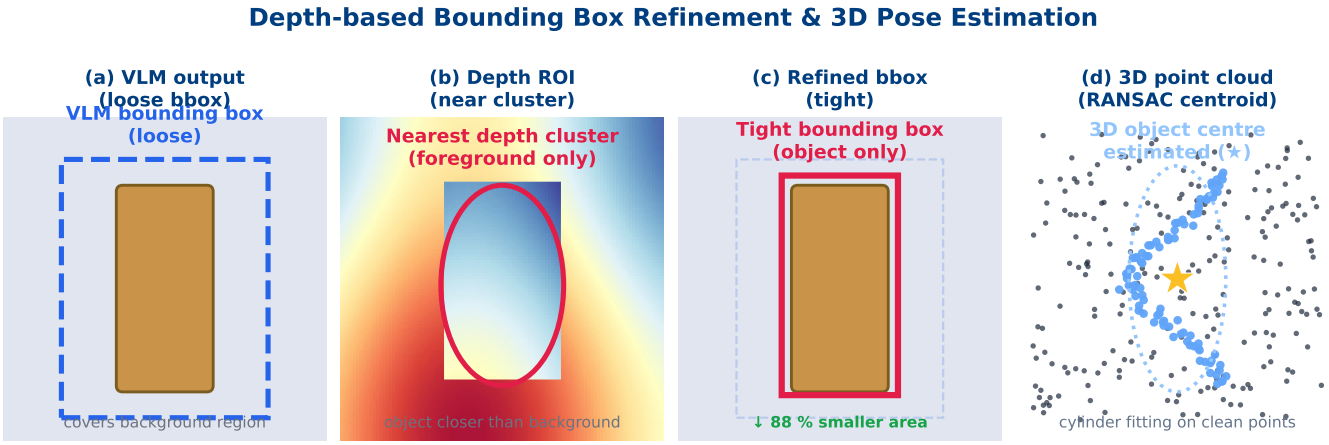

Figure 3: **Depth-based bounding box refinement.** The VLM produces a semantically correct but spatially loose bounding box (a). Depth pixels within that region are filtered to the nearest foreground cluster (b), yielding a tight box that encloses only the object (c). The resulting clean point cloud is fed to RANSAC cylinder fitting (d).

named locations. Navigation targets are hardcoded; replacing them with a semantic 3D map (e.g., SayPlan-style [5]) would support multi-room deployment without manual calibration. **API dependency.** Cloud VLM availability is a single point of failure. Deploying a quantised local model (LLaVA-7B or similar) would provide offline resilience, at the cost of higher latency and a GPU requirement. **Cluttered scenes.** The nearest-depth cluster assumption breaks when multiple objects occupy the same depth band, potentially merging two adjacent objects into a single refined bbox. Integrating instance segmentation — for example, the incremental learning module of Kenai et al. [7] — would resolve per-object boundaries in such cases. **Memory recency.** The 5-minute recency window for object memory is appropriate for a static tabletop, but too long for highly dynamic scenes where objects are frequently moved by a user. A shorter, task-adaptive timeout would better reflect actual scene state in interactive environments. **Evaluation scope.** The system has been demonstrated in a controlled laboratory environment with stable lighting; further testing in unstructured, changing environments (e.g., care homes, offices) is needed to assess real-world robustness.

6. Conclusion

We presented a neuro-symbolic embodied agent for the TIAGo mobile manipulator that closes the full loop from free-form voice commands to physical object handover, without a local GPU and without retraining for new object categories. The integration demonstrates three practical design choices: *(i)* a live semantic state manager (StateManager) that injects structured perceptual context into every VLM prompt, enabling session-persistent multi-instance disambiguation without weight updates; *(ii)* a single VLM call for both open-vocabulary detection and skill selection, replacing the separate detector + planner architecture of prior work; and *(iii)* a depth-based foreground extraction step that improves 3D localisation from loose VLM bounding boxes without segmentation masks.

The work builds on the LISSI line of neuro-symbolic systems [4, 6], extending it from static knowledge-assisted control to a fully deployed agent capable of physical manipulation and object handover on a real mobile platform. We hope this integration serves as a practical blueprint for deploying cloud-based VLMs in embodied robotics settings that do not have access to local GPU hardware.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, pages 287–318, 2022.

[2] Yingdong Hu, Fanqi Lin, Tong Zhang, Li Yi, and Yang Gao. Look before you leap: Unveiling the power of GPT-4V in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023.

[3] Francesco Argenziano, Michele Brienza, Vincenzo Suriani, Daniele Nardi, and Domenico D. Bloisi. EMPOWER: Embodied multi-role open-vocabulary planning with online grounding and execution. *arXiv preprint arXiv:2408.17379*, 2024.

[4] Oussama Eladdachi, Ferhat Attal, Abdelghani Chibani, Ilies Chibane, Imad-Eddine Kenai, and Yacine Amirat. Integrating LLM, semantic perception and spatial reasoning for improved robot action control. In *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, volume 413 of *Frontiers in*

Artificial Intelligence and Applications, pages 5179–5182. IOS Press, 2025. doi: 10.3233/FAIA251449.

[5] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Sünderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *7th Conference on Robot Learning (CoRL 2023)*, 2023.

[6] H. Abdelkawy, N. Ayari, Abdelghani Chibani, Yacine Amirat, and Ferhat Attal. Spatio-temporal convolutional networks and N-Ary ontologies for human activity-aware robotic system. *IEEE Robotics and Automation Letters*, 6(2):620–627, 2021. doi: 10.1109/LRA.2020.3047780.

[7] Imad Eddine Kenai, Abdelghani Chibani, Ferhat Attal, Ilies Chibane, and Yacine Amirat. Neuro-symbolic framework integrating incremental learning with LLM and symbolic reasoning on unknown objects. In *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, volume 413 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2025. doi: 10.3233/FAIA251451.